Esch384 Hash on FPGA: Design Document

ECE 545 Final Project • Henry Su, Nick Ngo • November 10, 2026

Goal

Build Esch384 in hardware on an FPGA. Esch384 is a hash function from the SPARKLE family, a finalist in NIST's Lightweight Cryptography competition. It takes a message of any length and gives a 384-bit (48-byte) digest. It is a *sponge*: the message goes in 16 bytes at a time (rate 128 bits), and the rest of the 512-bit state (capacity 384 bits) is never touched directly.

Design strategy

We build a simple *iterative* design first: one permutation step per clock cycle. We make it fully correct before we try to make it smaller. Hashing a message of n blocks takes $8n + 20$ steps (8 per block, 4 extra on the last block, and 16 to output the 3 digest pieces).

Components

Block	What it does
Alzette	Mixing box on two 32-bit words: add, rotate, XOR (4 rounds). 8 copies, one per branch.
Linear layer	Mixes the 8 branches together and swaps their order.
SPARKLE512	One step = constant + 8 Alzettes + linear layer; repeated 8 or 12 times.
Datapath	512-bit state register, message padding and injection, digest output.
Controller	State machine: take block, run steps, output 3 digest pieces; valid/ready handshake.
Top level	Datapath + controller, tested against all known-answer tests.

Tools and libraries

SystemVerilog (kept close to Verilog-2001); Xilinx Vivado 2026.1 for synthesis and xsim for simulation (Questa as backup); target board Arty A7 (Artix-7, `xc7a100tcsg324-1`). Reference: the official NIST C code for Esch384, plus Python scripts that create test vectors and compare outputs. No outside hardware libraries (IP cores). Code is on GitHub. **AI tool:** Claude Code (Anthropic). It drafts RTL, testbenches and scripts, and helps with debugging; we run every test ourselves before accepting its work. Our proposal named GPT 6 Astra; we switched to Claude Code. All AI tool use is logged in the repo (`docs/ai_log.md`).

Optimization target: area, and planned optimizations

Lightweight cryptography is about small area, so area is our target. For every version we report LUTs, flip-flops, maximum clock speed, latency and throughput.

- **Serialize the Alzettes:** use 4, 2 or 1 Alzette box instead of 8, and take more cycles per step, like the 32-bit serial AES datapath in lecture 13 (Aysu et al., ESL'14).
- Share adders, reuse registers, try a narrower datapath. One git branch per idea; all tests re-run after every change.

Risk management

- **Wrong results:** every block is tested on its own against the official C code, and we break a copy on purpose to make sure the test catches it. For bugs, the reference prints the state after every step, so we can find the first step that differs.
- **Schedule:** tasks are small and split between us. The working version gets a git tag before any optimization, so we can always go back to it.

Progress and plan

Item	Owner	Status / due	Result
Setup: tools, repo, course notes	Henry	Done	Vivado/xsim 2026.1 working
Reference model and test vectors	Henry	Done	Matches all 1025 official test vectors
Alzette	Henry	Done	64/64 tests pass; 254 LUTs
Linear layer	Henry	Done	64/64 tests pass; 320 LUTs
SPARKLE512 permutation	Henry	Done	71/71 tests pass; 2023 LUTs, 521 FFs; max clock 76 MHz
Datapath and controller	Nick	Nov 17	Specs and test plan written
Full hash, all test vectors	Nick	Nov 21	–
Synthesis: area, speed, throughput	Henry	Nov 24	–
Area optimization	Both	Dec 1	–
Final presentation and report	Both	Dec 9	–

LUT/FF counts are post-synthesis (Vivado, module on its own); 76 MHz is after place and route. 2023 is less than $8 \times 254 + 320$ because inside the permutation the constants are fixed, so Vivado simplifies them and packs the linear-layer XORs into the same LUTs as the Alzette logic. SPARKLE512 takes 9 / 13 cycles: 1 load cycle + 8 or 12 steps.

Challenges and how we solved them

- **Byte and word order were unclear in the spec.** We checked them against the official C code.
- **The simulator (xsim) crashed** on one testbench line. We rewrote the line a simpler way.